Bulletin Board System

Foundations of Cybersecurity/Applied Cryptography (a.a. 2023/2024)

Group members:

Andrea Fabbri
Giacomo Lombardi
Giacomo Maldarella

Summary

1. Structure of the prototype

The prototype of the BBS system consists of three source files: '*client.c*', '*server.c*', and '*utility.h*'. These files contain the implementation of client-side and server-side functionalities, as well as shared utilities between them.
The BBS is implemented in a centralized manner, with a BBS server located at a well-known (IP, port) pair. When the server is executed the port can be selected as parameter, while the IP address is 127.0.0.1.
The prototype uses a folder hierarchy for managing cryptographic keys:

- *keys_clients*: contains the private-public key pairs of clients.
- *keys_server*: contains the private-public key pairs of the server. Within this folder, there is a subfolder named *keys_retrieved*, which houses the public keys of clients that the server has obtained.
- *Download*: contains all the messages downloaded by the user using the get function.

Cryptographic algorithms employed:

- Digital signature: RSA
- Symmetric encryption: AES-256 in ECB mode
- MAC: HMAC SHA-256
- Hash function: SHA-256

2. Overview

The Bulletin Board System (BBS) allows users to read messages and add their own through the following operations:

- List: shows the latest n available messages in the BBS.
- Get: downloads a specific message identified by its unique identifier.
- Add: adds a new message to the system, specifying title, author, and body.

These operations can only be performed by a registered user after successful login. Every interaction between user and server is conducted within a secure session, ensuring that an attacker cannot read or tamper the communication. This means that messages are never transmitted in the clear and their integrity is always guaranteed.

3. Session

A session represent a period of interaction between a client and the server. It is characterized by the following parameters:

- Session Key for Secrecy (SessionKey1): used to ensure that the data exchanged during the session is encrypted and remains confidential.
- Session Key for Authenticity (SessionKey2): used to guarantee the integrity and authenticity of the data.
- Nonce: a unique value with multiple objectives:
    - After login, it is used, together with the password, to generate SessionKey2 as H(H(password) || nonce)
    - It serves as the starting point of the sequence number used to guarantee the freshness of messages.

Every session begins with the execution of a handshake.

4. Phases

The system is composed of four different phases:

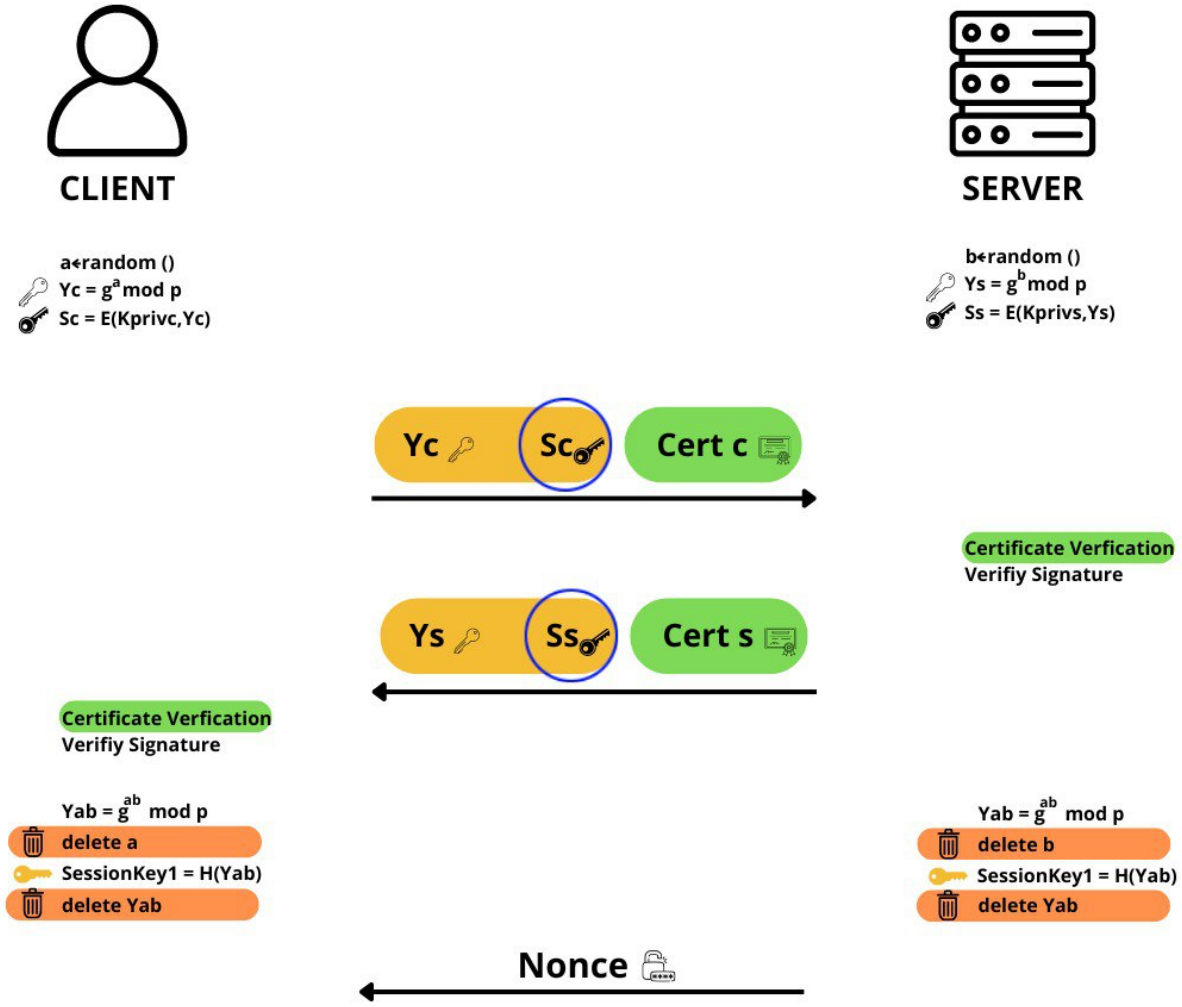
- Handshake: allows client and server to obtain a new SessionKey1 and a nonce. The handshake always precedes registration and login. The main difference between the two cases is that when followed by registration, the nonce won't be used.
- Registration: during registration, a session starts in which credentials (email address, username, and password) sent from the client to the server are encrypted with SessionKey1, while integrity is guaranteed through a digital signature.
  The session ends at the conclusion of the registration.
- Login: when login is performed, a new session starts in which credentials (username and password) are encrypted with SessionKey1. The difference from the registration phase is that, since both client and server know H(password), they can independently generate SessionKey2 and compute the MAC. A MAC has been chosen for integrity because it is more efficient than implementing a digital signature for each message.
- Application message exchange: the session lasts until logout, and all the application messages within the session are encrypted with SessionKey1 while integrity is guaranteed with the MAC. Additionally, all messages encapsulate a sequence number that allows the server and client to ensure their freshness.

In the following sections you will find a detailed description of each phase, emphasizing how confidentiality, integrity, non-replay, non-malleability and Perfect Forward Secrecy (PFS) are ensured.
The approach followed for each phase includes:

- Overview of the phase.
- Graphical representation.
- Main assumptions made during the design and implementation.
- Considerations on how different threats are managed.

5. Handshake

Clients and server are equipped with private-public key pairs, which are used to implement digital signature necessary to ensure integrity and avoid man-in-the-middle attacks.
The key exchange algorithm employed is Ephemeral Diffie-Hellman.



5.1 Assumptions

- This protocol operates under the assumption that both the client and server have securely obtained each other's public keys through certificates. In the prototype, this process is simulated as follows for simplicity:
  - Both the client and server generate their own private-public key pair and place them respectively in the *keys_clients* and *keys_server* folders.
  - It is assumed that the client knows the server's public key beforehand, so the client retrieves this key from the *keys_server* folder at the beginning of the handshake.
  - The client sends its public key to the server, simulating the act of sending a certificate signed by a Certification Authority (CA).

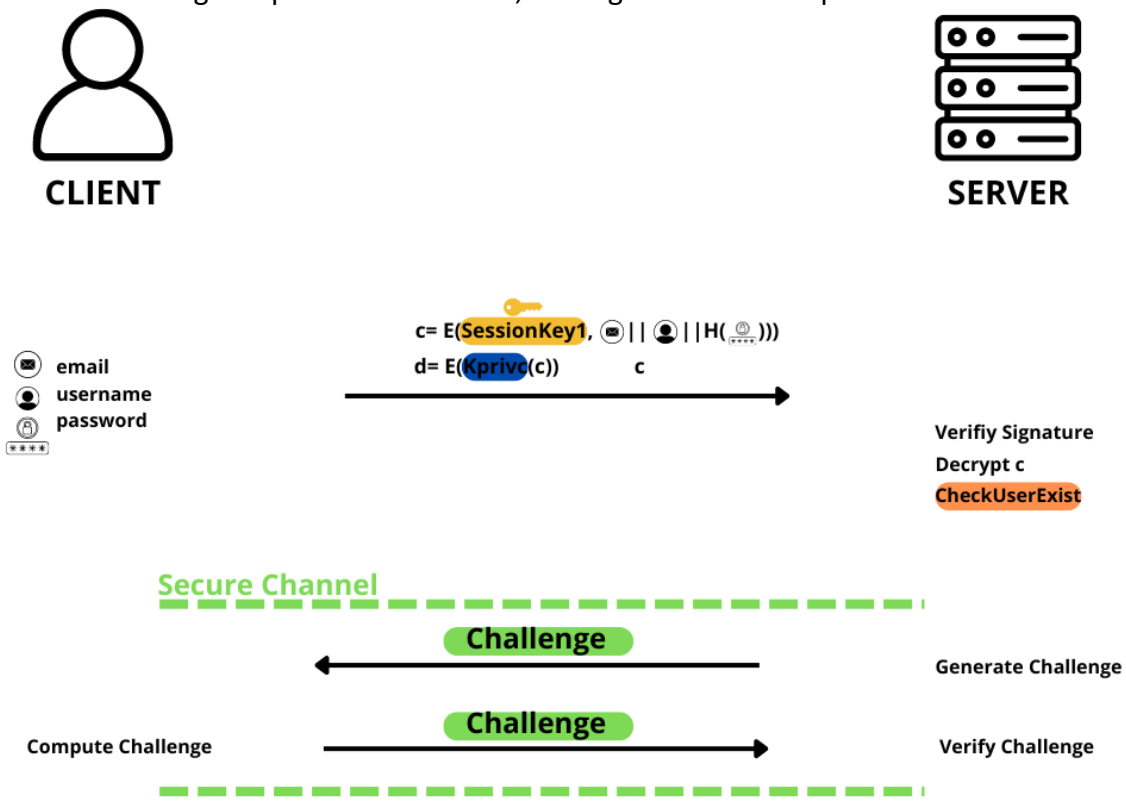
5.2 Security

- Replay Attack: this threat is mitigated by the fact that secret parameters a and b are freshly generated for each handshake execution. As a result, replaying $[g^a, E(Kpriv_c, g^a)]$ or $[g^b, E(Kpriv_S, g^b)]$ would yield different session key values.

- Man-in-the-Middle Attack (MIM): to mitigate this attack, both parties sign their respective public Diffie-Hellman parameters. Therefore, a MIM attack is not feasible due to the requirement for an adversary to sign g^c with the private keys of client and server.
- Perfect Forward Secrecy (PFS): after key generation, parameters a, b, and g^{ab} are discarded. Consequently, an adversary intercepting the communication cannot derive the session key as it requires to solve the Discrete Logarithm Problem (DLP).

6. Registration

The registration process enables a user to securely transmit email address, username, and password to the BBS server.
Upon receiving the encrypted credentials and signature from the client, the server verifies the integrity of the data through signature verification. In this context, integrity is ensured through a digital signature, as the server does not yet have the necessary parameters to generate the key used for computing a MAC.
Once validated, the ciphertext is decrypted using SessionKey1 to extract the user's credentials. The server then checks the uniqueness of the provided username and email address to prevent duplicate registrations.
Finally, the server generates a random challenge and sends it to the client over a secondary secure channel. If the challenge response is successful, the registration is completed.



6.1 Assumptions

- The implementation of the challenge-response mechanism over a secure channel is simplified as follows: the server generates a random challenge and writes it into a file accessible to the client. The client then reads the challenge from this file and sends it back to the server for verification.
- In principle, user's credentials should be stored in a file. However, to simplify the implementation of the prototype, the credentials are kept in a data structure and never transferred into a file. The main drawback of this simple solution is that when the server is closed, all information is lost.

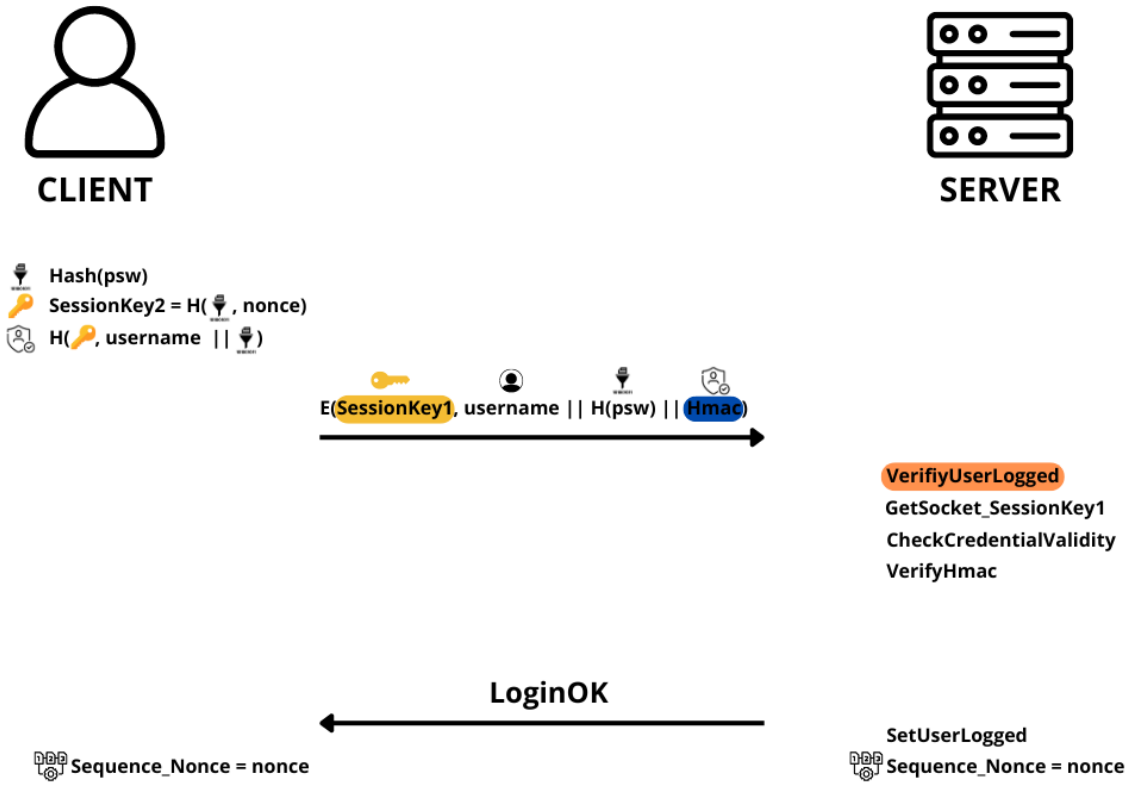
6.2 Security

- Confidentiality: user credentials are encrypted using SessionKey1 to ensure that sensitive information remains confidential during transmission.

- Integrity: the integrity of the transmitted data is maintained through the use of digital signature applied to the ciphertext. This ensures that the received data has not been tampered with during transit.
- Replay Attacks: once a user has been successfully registered, the system prevents the replay of registration attempts. This safeguard ensures that duplicate credentials cannot be used to register additional accounts.

7. Login

A registered user logins by means of his/her username and password. This phase uses two main security parameters: SessionKey1 and SessionKey2.
To avoid storing and sending the password in clear text, the client computes and stores the hash of the password, which is later used to generate SessionKey2. To protect both confidentiality and integrity, a MAC then Encrypt scheme is employed. The client sends the message encrypted with SessionKey1: Enc(SessionKey1, m), where m = (username || H(password) || HMAC). HMAC is computed using SessionKey2.
Upon receiving the ciphertext, the server checks if the user is already logged in and then proceeds to decrypt the ciphertext and verify if the credentials sent correspond to the credentials stored in the registration phase. The last check involves MAC verification to ensure integrity/authentication. If the login is successful, the server and client initialize the sequence number to a random value, and the sever sets the variable '*is_logged*' to true.



7.1 Assumptions

- A secure implementation of the login phase necessitates that a client cannot submit an infinite number of passwords to attempt accessing the application. However, for testing purposes, in the prototype, the server does not make any check on the number of login attempts.

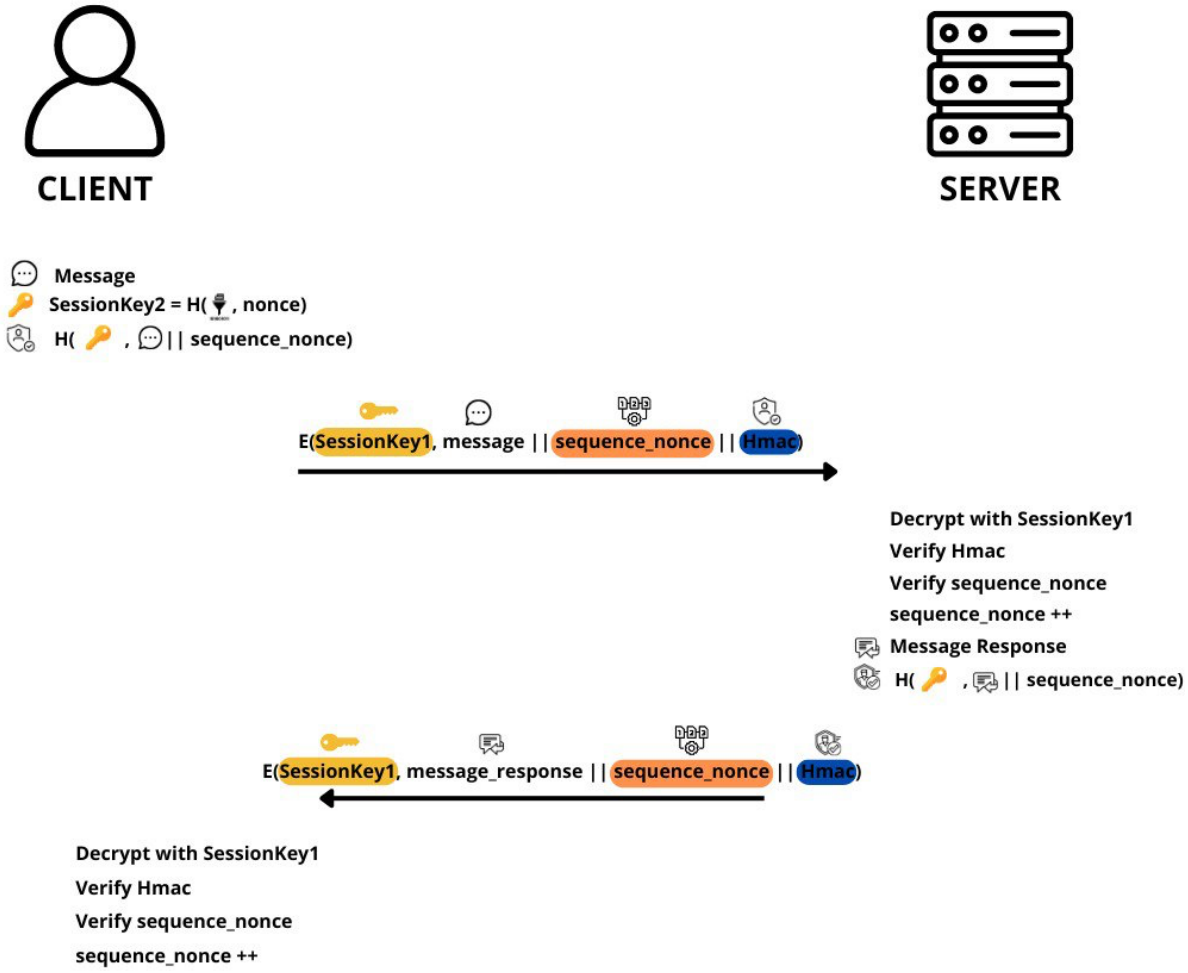
7.2 Security

- Confidentiality: user credentials are encrypted using SessionKey1.
- Integrity/authentication: the server verifies the integrity of the received message by decrypting it and performing MAC verification. Additionally, the server compares the credentials with those stored during the registration phase.

- Non-replay: to prevent replay attacks during the login phase, a variable '*is_logged*' is associated to the socket from which the client is connected. The server checks if it is false when it receives the message containing the user's credentials.

8. Application message exchange

At this stage, all communications between a logged client and the server take place on a secure channel. The client's messages pertain to one of the four possible commands (list, add, get, logout), while the server responds whit the specific services.
For each message to be sent, client and server compute the HMAC of the concatenation between the application message and the sequence number. After that, they send the encryption of the message, the sequence number, and the MAC. In the prototype, all these operations are encapsulated in a function named '*messageDelivery()*'. If everything goes right, the sequence number is incremented.
The receiving phase involves the decryption of the ciphertext, the verification of the MAC, and the verification of the sequence number. In the prototype, these operations are encapsulated in a function named '*messageReceipts()*'. If the checks pass, the sequence number is incremented.



8.1 Assumptions

- To better manage the body of messages, which can be large and contain many white spaces, it has been decided to encode them in their hexadecimal equivalent.

8.2 Security

- Confidentiality: all messages are encrypted with SessionKey1.
- Integrity/authenticity: it is ensured through the use of HMAC.
- Non-reply: the sequence number attached to the messages prevents replay attacks by ensuring that each message is unique.

9. Advantages and disadvantages of the implementation

9.1 Advantages

- The system is secure against MIM attacks since the public keys are exchanged using certificates.
- The system is secure against replay attack in all its phases:
    - The handshake is protected against replay thanks to the use of Ephemeral DH.
    - The registration phase is protected from this replay attacks because the server checks if a user is already registered.
    - The variable '*is_logged*' and the sequence number protect the login phase and all subsequent messages from replay attacks.
- If an adversary is able to compromise both session keys, the damages are limited since SessionKey1 and SessionKey2 are new for each session. Moreover, Perfect Forward Secrecy (PFS) is ensured thanks to the Discrete Logarithm Problem difficulty.
- Even if SessionKey1 is compromised, an adversary is unable to forge a MAC using the server as an oracle. If the MAC does not match, the server closes the connection, and the keys are changed.

9.2 Disadvantages

- The handshake protocol does not fulfill the property of Direct Authentication since there are no guarantees that both the client and server own the keys. They need to wait for the first application message to ensure key confirmation.
- Passwords are managed in hashed form; therefore, even if an adversary is able to obtain the list of passwords, s(he) won't be able to read them. However, this method of password management has the following drawbacks:
    - Duplicated passwords are visible since their hashes are equal.
    - Offline attacks such as Dictionary and Rainbow table attacks are possible.

  To mitigate these risks, a better solution is to use a salt concatenated to the password.
- In the prototype, hexadecimal codification is used to send hashes, HMACs, and application messages. This solution decreases performance since buffer sizes are doubled.
- The symmetric encryption scheme employed is AES together with ECB, which generally has vulnerabilities such as block reordering and substitution, traffic and pattern analysis. However, considering the introduction of nonces, digital signatures, and MACs, these problems are not a significant concern for this application.